Access と同じ年金業務表を N2 JNDI 経由で操作する Repository

表紙

項目	内容
文書名	基本設計書
システム名	
機能ID	COM-145
画面名	Access と同じ年金業務表を N2 JNDI 経由で操作する Repository
会社名	
作成日	

概要

概要説明

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryの入力・表示・要求処理を行う。

情報	内容
対象種別	common

情報	内容
開始契機	呼出元からの要求
処理対象	DB2.Ｕ職員名簿表、N2_SIGHT.Ｂ顧客基本表、受給有無テーブル、年金1_LK、年金種類テーブル、年金__削除明細、店名テーブル、性別テーブル、裁定請求テーブル

項目一覧

No	グループ	項目名	種類	入出力	DB連携項目	候補取得元	必須	形式・桁	説明

項目定義数：0項目。表示順と反復行の定義数を区別する。

イベント仕様

PensionRepository

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「PensionRepository」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連する処理へ制御を渡す。詳細な実行経路は原本対応を参照する。

getInstance

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「getInstance」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

createEmpty

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「createEmpty」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

createEmpty

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「createEmpty」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

findByCustomer

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「findByCustomer」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「C WHERE C.店舗コード = ? AND C.顧客番号 = ?」を設定する。
2. 関連情報を取得・設定する（処理14：詳細仕様参照）。
3. 関連情報を取得・設定する（処理15：詳細仕様参照）。
4. 関連情報を取得・設定する（処理16：詳細仕様参照）。
5. 関連情報を取得・設定する（処理17：詳細仕様参照）。
6. 関連情報を取得・設定する（処理18：詳細仕様参照）。
7. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

7.1. 通常処理：関連情報を取得・設定する（処理19：詳細仕様参照）。

7.2. 通常処理：関連情報を取得・設定する（処理20：詳細仕様参照）。

7.3. 通常処理：「対象項目」を設定する。

7.4. 通常処理：「対象項目」を設定する。

7.5. 通常処理：関連情報を取得・設定する（処理21：詳細仕様参照）。

7.6. 通常処理：「条件22を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

7.6.1. 条件成立時：関連情報を取得・設定する（処理23：詳細仕様参照）。

7.6.2. 条件成立時：「対象項目」を設定する。

7.6.3. 条件成立時：「対象項目」を設定する。

7.6.4. 条件成立時：「対象項目」を設定する。

7.6.5. 条件成立時：「対象項目」を設定する。

7.6.6. 条件成立時：「対象項目」を設定する。

7.6.7. 条件成立時：関連情報を取得・設定する（処理24：詳細仕様参照）。

7.6.8. 条件成立時：「対象項目」を設定する。

7.6.9. 条件成立時：「対象項目」を設定する。

7.6.10. 条件成立時：「対象項目」を設定する。

7-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

7-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

7-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

7-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

7-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

7-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

1. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

8.1. 条件成立時：戻り先画面を指定せず処理を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理25：詳細仕様参照）。
2. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

10.1. 通常処理：関連情報を取得・設定する（処理26：詳細仕様参照）。

10.2. 通常処理：「店舗名」を設定する。

10.3. 通常処理：「対象項目」を設定する。

10.4. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

10-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

10-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

10-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

10-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-26B9F21A0A7B

情報	内容
操作ID	SQL-26B9F21A0A7B
対象テーブル	N2_SIGHT.B顧客基本表
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：C.店舗コード = ? AND C.顧客番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:88 / findByCustomer@87

DB操作 SQL-22BD3A50DAE5

情報	内容
操作ID	SQL-22BD3A50DAE5

情報	内容
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：店舗ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:543 / resolveBranchName@535

結果・例外時の扱い

到達条件	結果
条件27を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
対象項目がない	戻り先画面を指定せず処理を終了する
NOT (value が等しい 未設定)	処理結果を呼出元へ返す
NOT (value が等しい 未設定)・条件28を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

findByKanaAndPhone

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「findByKanaAndPhone」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理9：詳細仕様参照）。
2. 関連情報を取得・設定する（処理10：詳細仕様参照）。
3. 「kana.length() が等しい 0、または対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：戻り先画面を指定せず処理を終了する。この経路の後続手順は実行しない。

1. 「WHERE R.ふりがな = ? AND R.電話番号 = ?」を設定する。
2. 関連情報を取得・設定する（処理11：詳細仕様参照）。
3. 関連情報を取得・設定する（処理12：詳細仕様参照）。
4. 関連情報を取得・設定する（処理13：詳細仕様参照）。
5. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

8.1. 通常処理：関連情報を取得・設定する（処理14：詳細仕様参照）。

8.2. 通常処理：関連情報を取得・設定する（処理15：詳細仕様参照）。

8.3. 通常処理：「対象項目」を設定する。

8.4. 通常処理：「対象項目」を設定する。

8.5. 通常処理：関連情報を取得・設定する（処理16：詳細仕様参照）。

8.6. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

8-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

8-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

8-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

8-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

8-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

8-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-BE29DD398BAD

情報	内容
操作ID	SQL-BE29DD398BAD
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:150 / findByKanaAndPhone@144

DB操作 SQL-98675681E706

情報	内容
操作ID	SQL-98675681E706
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:150 / findByKanaAndPhone@144

DB操作 SQL-935A991E07F4

情報	内容
操作ID	SQL-935A991E07F4
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:150 / findByKanaAndPhone@144

DB操作 SQL-A0A469ECF2C0

情報	内容
操作ID	SQL-A0A469ECF2C0
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:150 / findByKanaAndPhone@144

DB操作 SQL-F2C97CD89E50

情報	内容
操作ID	SQL-F2C97CD89E50
対象テーブル	受給有無テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:150 / findByKanaAndPhone@144

DB操作 SQL-CD804AC01D11

情報	内容
操作ID	SQL-CD804AC01D11
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:150 / findByKanaAndPhone@144

結果・例外時の扱い

到達条件	結果
kana.length() が等しい 0、または対象項目がない	戻り先画面を指定せず処理を終了する
NOT (kana.length() が等しい 0 || phone が等しい 未設定)	処理結果を呼出元へ返す
NOT (kana.length() が等しい 0 || phone が等しい 未設定)・条件17を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

searchCandidates

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「searchCandidates」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理16：詳細仕様参照）。
2. 関連情報を取得・設定する（処理17：詳細仕様参照）。
3. 「R.生年月日 AS BIRTH_DATE FROM 」を設定する。
4. 「条件18を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.1. 条件成立時：「 AND UPPER(R.ふりがな) LIKE UPPER(?)」を設定する。

1. 「条件19を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1. 条件成立時：「 AND CAST(R.電話番号 AS VARCHAR(40)) LIKE ?」を設定する。

1. 「 ORDER BY R.ふりがな, R.店番, R.顧客番号」を設定する。
2. 関連情報を取得・設定する（処理20：詳細仕様参照）。
3. 関連情報を取得・設定する（処理21：詳細仕様参照）。
4. 関連情報を取得・設定する（処理22：詳細仕様参照）。
5. 関連情報を取得・設定する（処理23：詳細仕様参照）。
6. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

11.1. 通常処理：関連情報を取得・設定する（処理24：詳細仕様参照）。

11.2. 通常処理：関連情報を取得・設定する（処理25：詳細仕様参照）。

11.3. 通常処理：関連情報を取得・設定する（処理26：詳細仕様参照）。

11.4. 通常処理：「条件27を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

11.4.1. 条件成立時：「対象項目」を設定する。

11.5. 通常処理：「条件28を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

11.5.1. 条件成立時：「対象項目」を設定する。

11.6. 通常処理：関連情報を取得・設定する（処理29：詳細仕様参照）。

11.7. 通常処理：繰返し条件「条件30を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

11.7.1. 繰返しの各回：年金予約候補データを実行する。

11.8. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

11-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

11-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

11-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

11-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

11-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

11-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件31を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

list

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「list」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

listByBranch

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「listByBranch」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「 WHERE R.店番 = ? ORDER BY R.請求予定日 DESC, R.担当者コード, R.顧客番号」を設定する。
2. 関連情報を取得・設定する（処理10：詳細仕様参照）。
3. 関連情報を取得・設定する（処理11：詳細仕様参照）。
4. 関連情報を取得・設定する（処理12：詳細仕様参照）。
5. 関連情報を取得・設定する（処理13：詳細仕様参照）。
6. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

6.1. 通常処理：関連情報を取得・設定する（処理14：詳細仕様参照）。

6.2. 通常処理：関連情報を取得・設定する（処理15：詳細仕様参照）。

6.3. 通常処理：「対象項目」を設定する。

6.4. 通常処理：関連情報を取得・設定する（処理16：詳細仕様参照）。

6.5. 通常処理：繰返し条件「条件17を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

6.5.1. 繰返しの各回：関連情報を取得・設定する（処理18：詳細仕様参照）。

6.6. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

6-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

6-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

6-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

6-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

6-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

6-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-1CC29D9EDFFF

情報	内容
操作ID	SQL-1CC29D9EDFFF
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.店番 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:224 / listByBranch@223

DB操作 SQL-28470D010AFF

情報	内容
操作ID	SQL-28470D010AFF
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.店番 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:224 / listByBranch@223

DB操作 SQL-0F5F7D3D4469

情報	内容
操作ID	SQL-0F5F7D3D4469
対象テーブル	性別テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.店番 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:224 / listByBranch@223

DB操作 SQL-0E90141F3DE0

情報	内容
操作ID	SQL-0E90141F3DE0
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.店番 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:224 / listByBranch@223

DB操作 SQL-6597FFB0C483

情報	内容
操作ID	SQL-6597FFB0C483
対象テーブル	受給有無テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.店番 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:224 / listByBranch@223

DB操作 SQL-87035C2BB1E0

情報	内容
操作ID	SQL-87035C2BB1E0
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.店番 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:224 / listByBranch@223

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件19を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

search

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「search」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理3：詳細仕様参照）。
2. 「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：「 AND R.店番 >= ?」を設定する。

1. 「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：「 AND R.店番 <= ?」を設定する。

1. 値に応じた分岐の詳細は原本対応を参照する。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

save

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「save」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-22BD3A50DAE5

情報	内容
操作ID	SQL-22BD3A50DAE5
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：店舗ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:543 / resolveBranchName@535

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-19BE5BBBA554

情報	内容
操作ID	SQL-19BE5BBBA554
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:628 / exists@623

DB操作 SQL-1A463E2BC06C

情報	内容
操作ID	SQL-1A463E2BC06C
対象テーブル	年金1_LK
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）

情報	内容
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:661 / insert@657

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0

設定先のDB項目	バインド順	値の設定・変換（原本）
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, Integer.valueOf(0))

DB操作 SQL-CD4068C5AE3B

情報	内容
操作ID	SQL-CD4068C5AE3B
対象テーブル	年金1_LK
操作	更新
処理説明	対象テーブルの条件に一致する行を更新する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）

情報	内容
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:676 / update@672

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0

設定先のDB項目	バインド順	値の設定・変換（原本）
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, Integer.valueOf(1))

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

save

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「save」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理11：詳細仕様参照）。
2. 関連情報を取得・設定する（処理12：詳細仕様参照）。
3. 関連情報を取得・設定する（処理13：詳細仕様参照）。
4. 関連情報を取得・設定する（処理14：詳細仕様参照）。
5. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

5.1. 通常処理：関連情報を取得・設定する（処理15：詳細仕様参照）。

5.2. 通常処理：変更内容の確定を明示的に制御する。

5.3. 通常処理：「店舗名」を設定する。

5.4. 通常処理：関連情報を取得・設定する（処理16：詳細仕様参照）。

5.5. 通常処理：関連情報を取得・設定する（処理17：詳細仕様参照）。

5.6. 通常処理：関連情報を取得・設定する（処理18：詳細仕様参照）。

5.7. 通常処理：「条件19を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

5.7.1. 条件成立時：対象データを更新する。

5.7-別.1. 条件不成立時：新しいデータを登録する。

5.8. 通常処理：変更内容を確定する（コミット）。

5.9. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

5-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

5-例外1.1. 例外発生時：このトランザクションの変更を取り消す（ロールバック）。

5-例外1.2. 例外発生時：「条件20を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5-例外1.2.1. 条件成立時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

5-例外1.3. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

5-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

5-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-22BD3A50DAE5

情報	内容
操作ID	SQL-22BD3A50DAE5
対象テーブル	店名テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：店舗ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:543 / resolveBranchName@535

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-19BE5BBBA554

情報	内容
操作ID	SQL-19BE5BBBA554
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:628 / exists@623

DB操作 SQL-1A463E2BC06C

情報	内容
操作ID	SQL-1A463E2BC06C
対象テーブル	年金1_LK
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:661 / insert@657

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, Integer.valueOf(0))

DB操作 SQL-CD4068C5AE3B

情報	内容
操作ID	SQL-CD4068C5AE3B
対象テーブル	年金1_LK
操作	更新
処理説明	対象テーブルの条件に一致する行を更新する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:676 / update@672

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, Integer.valueOf(1))

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件21を満たす（原本条件は補足説明を参照）・条件22を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件23を満たす（原本条件は補足説明を参照）・条件24を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

delete

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「delete」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

delete

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「delete」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理8：詳細仕様参照）。
2. 関連情報を取得・設定する（処理9：詳細仕様参照）。
3. 「kana.length() が等しい 0、または対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：不成立の結果を返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 関連情報を取得・設定する（処理11：詳細仕様参照）。
3. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

6.1. 通常処理：関連情報を取得・設定する（処理12：詳細仕様参照）。

6.2. 通常処理：変更内容の確定を明示的に制御する。

6.3. 通常処理：指定した条件でデータを照会する（照会結果1）。

6.4. 通常処理：「照会結果1がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

6.4.1. 条件成立時：このトランザクションの変更を取り消す（ロールバック）。

6.4.2. 条件成立時：終了時の共通後処理を行ってから、不成立の結果を返す。この経路の後続手順は実行しない。

6.5. 通常処理：「対象項目」を設定する。

6.6. 通常処理：関連情報を取得・設定する（処理13：詳細仕様参照）。

6.7. 通常処理：「 WHERE ふりがな = ? AND 電話番号 = ?」を設定する。

6.8. 通常処理：「対象項目」を設定する。

6.9. 通常処理：「対象項目」を設定する。

6.10. 通常処理：関連情報を取得・設定する（処理14：詳細仕様参照）。

6.11. 通常処理：「deleted が異なる 1」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

6.11.1. 条件成立時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

6.12. 通常処理：変更内容を確定する（コミット）。

6.13. 通常処理：終了時の共通後処理を行ってから、成立の結果を返す。この経路の後続手順は実行しない。

6-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

6-例外1.1. 例外発生時：このトランザクションの変更を取り消す（ロールバック）。

6-例外1.2. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

6-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

6-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

6-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）

情報	内容
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0

設定先のDB項目	バインド順	値の設定・変換（原本）
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
kana.length() が等しい 0、または対象項目がない	不成立の結果を返す
NOT (kana.length() が等しい 0 || phone が等しい 未設定)・照会結果1がない	不成立の結果を返す
NOT (kana.length() が等しい 0 || phone が等しい 未設定)・NOT (existing が等しい 未設定)・deleted が異なる 1	処理の異常を通知し、この経路を終了する
NOT (kana.length() が等しい 0 || phone が等しい 未設定)・NOT (existing が等しい 未設定)・NOT (deleted が異なる 1)	成立の結果を返す
NOT (kana.length() が等しい 0 || phone が等しい 未設定)・条件15を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

findAuditRecords

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「findAuditRecords」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理19：詳細仕様参照）。
2. 関連情報を取得・設定する（処理20：詳細仕様参照）。

3. 関連情報を取得・設定する（処理21：詳細仕様参照）。

4. 「R.更新者 AS UPDATED_BY」を設定する。

5. 「type が異なる AuditType.DELETED」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

6.1. 条件成立時：「 AND R.変更FLG = ?」を設定する。

1. 「条件22を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

7.1. 条件成立時：「 AND R.データ更新日 >= ?」を設定する。

1. 「条件23を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

8.1. 条件成立時：「 AND R.データ更新日 <= ?」を設定する。

1. 「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

9.1. 条件成立時：「 AND R.更新者 = ?」を設定する。

1. 「 ORDER BY R.データ更新日, R.更新時間, R.店番, R.顧客番号」を設定する。

2. 関連情報を取得・設定する（処理24：詳細仕様参照）。

3. 関連情報を取得・設定する（処理25：詳細仕様参照）。

4. 関連情報を取得・設定する（処理26：詳細仕様参照）。

5. 関連情報を取得・設定する（処理27：詳細仕様参照）。

6. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

15.1. 通常処理：関連情報を取得・設定する（処理28：詳細仕様参照）。

15.2. 通常処理：関連情報を取得・設定する（処理29：詳細仕様参照）。

15.3. 通常処理：関連情報を取得・設定する（処理30：詳細仕様参照）。

15.4. 通常処理：「type が異なる AuditType.DELETED」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

15.4.1. 条件成立時：「対象項目」を設定する。

15.5. 通常処理：「条件31を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

15.5.1. 条件成立時：「対象項目」を設定する。

15.6. 通常処理：「条件32を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

15.6.1. 条件成立時：「対象項目」を設定する。

15.7. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

15.7.1. 条件成立時：「対象項目」を設定する。

15.8. 通常処理：関連情報を取得・設定する（処理33：詳細仕様参照）。

15.9. 通常処理：関連情報を取得・設定する（処理34：詳細仕様参照）。

15.10. 通常処理：繰返し条件「条件35を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

15.10.1. 繰返しの各回：関連情報を取得・設定する（処理36：詳細仕様参照）。

15.11. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

15-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

15-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

15-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

15-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

15-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

15-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

結果・例外時の扱い

到達条件	結果
対象項目がない	処理の異常を通知し、この経路を終了する
NOT (type が等しい 未設定)	処理結果を呼出元へ返す
NOT (type が等しい 未設定)・条件37を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

findLatestAuditType

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「findLatestAuditType」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理12：詳細仕様参照）。
2. 関連情報を取得・設定する（処理13：詳細仕様参照）。
3. 「kana.length() が等しい 0、または対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：戻り先画面を指定せず処理を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理14：詳細仕様参照）。
2. 関連情報を取得・設定する（処理15：詳細仕様参照）。
3. 関連情報を取得・設定する（処理16：詳細仕様参照）。
4. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

7.1. 通常処理：関連情報を取得・設定する（処理17：詳細仕様参照）。

7.2. 通常処理：「 WHERE ふりがな = ? AND 電話番号 = ?」を設定する。

7.3. 通常処理：「対象項目」を設定する。

7.4. 通常処理：「対象項目」を設定する。

7.5. 通常処理：「対象項目」を設定する。

7.6. 通常処理：関連情報を取得・設定する（処理18：詳細仕様参照）。

7.7. 通常処理：「条件19を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

7.7.1. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

7.7.2. 条件成立時：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

7.8. 通常処理：使用した接続・処理資源を解放する。

7.9. 通常処理：関連情報を取得・設定する（処理21：詳細仕様参照）。

7.10. 通常処理：使用した接続・処理資源を解放する。

7.11. 通常処理：「 ORDER BY データ更新日 DESC, 更新時間 DESC」を設定する。

7.12. 通常処理：「対象項目」を設定する。

7.13. 通常処理：「対象項目」を設定する。

7.14. 通常処理：「対象項目」を設定する。

7.15. 通常処理：関連情報を取得・設定する（処理22：詳細仕様参照）。

7.16. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

7-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

7-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

7-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

7-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

7-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

7-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-749C63561641

情報	内容
操作ID	SQL-749C63561641
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:427 / findLatestAuditType@416

DB操作 SQL-0E9B648B03D3

情報	内容
操作ID	SQL-0E9B648B03D3
対象テーブル	年金__削除明細
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:440 / findLatestAuditType@416

結果・例外時の扱い

到達条件	結果
kana.length() が等しい 0、または対象項目がない	戻り先画面を指定せず処理を終了する
NOT (kana.length() が等しい 0 || phone が等しい 未設定)・条件23を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (kana.length() が等しい 0 || phone が等しい 未設定)・条件24を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (kana.length() が等しい 0 || phone が等しい 未設定)・条件25を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

purgeDeletedAuditsThrough

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「purgeDeletedAuditsThrough」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 「cutoff.length() が等しい 0」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理7：詳細仕様参照）。
2. 関連情報を取得・設定する（処理8：詳細仕様参照）。
3. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

5.1. 通常処理：関連情報を取得・設定する（処理9：詳細仕様参照）。

5.2. 通常処理：変更内容の確定を明示的に制御する。

5.3. 通常処理：「 WHERE データ更新日 <= ?」を設定する。

5.4. 通常処理：「対象項目」を設定する。

5.5. 通常処理：関連情報を取得・設定する（処理10：詳細仕様参照）。

5.6. 通常処理：変更内容を確定する（コミット）。

5.7. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

5-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

5-例外1.1. 例外発生時：このトランザクションの変更を取り消す（ロールバック）。

5-例外1.2. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

5-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

5-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

5-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-4F7E42ED747E

情報	内容
操作ID	SQL-4F7E42ED747E
対象テーブル	年金__削除明細
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：データ更新日 <= ?
確定・取消	当該メソッド内で自動コミット設定・コミットを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:468 / purgeDeletedAuditsThrough@458

結果・例外時の扱い

到達条件	結果
cutoff.length() が等しい 0	処理結果を呼出元へ返す
NOT (cutoff.length() が等しい 0)	処理結果を呼出元へ返す
NOT (cutoff.length() が等しい 0)・条件11を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

queryReservations

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「queryReservations」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理11：詳細仕様参照）。
2. 関連情報を取得・設定する（処理12：詳細仕様参照）。
3. 関連情報を取得・設定する（処理13：詳細仕様参照）。
4. 関連情報を取得・設定する（処理14：詳細仕様参照）。
5. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

5.1. 通常処理：関連情報を取得・設定する（処理15：詳細仕様参照）。

5.2. 通常処理：関連情報を取得・設定する（処理16：詳細仕様参照）。

5.3. 通常処理：関連情報を取得・設定する（処理17：詳細仕様参照）。

5.4. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.4.1. 条件成立時：「対象項目」を設定する。

5.5. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.5.1. 条件成立時：「対象項目」を設定する。

5.6. 通常処理：関連情報を取得・設定する（処理18：詳細仕様参照）。

5.7. 通常処理：繰返し条件「条件19を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.7.1. 繰返しの各回：関連情報を取得・設定する（処理20：詳細仕様参照）。

5.8. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

5-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

5-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

5-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

5-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

5-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

5-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件21を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

prepareForSave

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「prepareForSave」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 「対象項目」を設定する。
3. 「対象項目」を設定する。
4. 「「対象項目」が空」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「phoneValue(「対象項目」) が等しい 未設定」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

6.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 入力値の制約と変換結果を確認する。
2. 「対象項目」を設定する。
3. 「対象項目」を設定する。
4. 「対象項目」を設定する。
5. 「対象項目」を設定する。
6. 「対象項目」を設定する。

7. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がない	処理の異常を通知し、この経路を終了する
NOT (source が等しい 未設定)・「対象項目」が空	処理の異常を通知し、この経路を終了する
NOT (source が等しい 未設定)・NOT (「対象項目」.length() が等しい 0)・phoneValue(「対象項目」) が等しい 未設定	処理の異常を通知し、この経路を終了する
NOT (source が等しい 未設定)・NOT (「対象項目」.length() が等しい 0)・NOT (phoneValue(「対象項目」) が等しい 未設定)	処理結果を呼出元へ返す

resolveBranchName

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「resolveBranchName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理4：詳細仕様参照）。
2. 関連情報を取得・設定する（処理5：詳細仕様参照）。

3. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4.1. 通常処理：「WHERE 店舗ｺｰﾄﾞ = ?」を設定する。

4.2. 通常処理：「対象項目」を設定する。

4.3. 通常処理：関連情報を取得・設定する（処理6：詳細仕様参照）。

4.4. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

4-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

4-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

4-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-22BD3A50DAE5

情報	内容
操作ID	SQL-22BD3A50DAE5
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：店舗ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。

情報	内容
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:543 / resolveBranchName@535

結果・例外時の扱い

到達条件	結果
対象項目がない	処理結果を呼出元へ返す
NOT (branchNo が等しい 未設定)	処理結果を呼出元へ返す

resolveCodeLabel

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「resolveCodeLabel」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理5：詳細仕様参照）。
2. 関連情報を取得・設定する（処理6：詳細仕様参照）。
3. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4.1. 通常処理：関連情報を取得・設定する（処理7：詳細仕様参照）。

4.2. 通常処理：「対象項目」を設定する。

4.3. 通常処理：関連情報を取得・設定する（処理8：詳細仕様参照）。

4.4. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

4-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

4-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

4-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

結果・例外時の扱い

到達条件	結果
対象項目がない	処理結果を呼出元へ返す
NOT (code が等しい 未設定)	処理結果を呼出元へ返す

resolveCode

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「resolveCode」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理7：詳細仕様参照）。
2. 「value.length() が等しい 0」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：戻り先画面を指定せず処理を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理8：詳細仕様参照）。
2. 関連情報を取得・設定する（処理9：詳細仕様参照）。
3. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

5.1. 通常処理：関連情報を取得・設定する（処理10：詳細仕様参照）。

5.2. 通常処理：「対象項目」を設定する。

5.3. 通常処理：関連情報を取得・設定する（処理11：詳細仕様参照）。

5.4. 通常処理：「条件12を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.4.1. 条件成立時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

5.5. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

5-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

5-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

5-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

結果・例外時の扱い

到達条件	結果
value.length() が等しい 0	戻り先画面を指定せず処理を終了する
NOT (value.length() が等しい 0)・条件13を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (value.length() が等しい 0)・条件14を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す

resolvePensionTypeCode

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「resolvePensionTypeCode」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 「code が等しい 0」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：戻り先画面を指定せず処理を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理7：詳細仕様参照）。
2. 関連情報を取得・設定する（処理8：詳細仕様参照）。
3. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

5.1. 通常処理：「 WHERE 種類ｺｰﾄﾞ = ?」を設定する。

5.2. 通常処理：「対象項目」を設定する。

5.3. 通常処理：関連情報を取得・設定する（処理9：詳細仕様参照）。

5.4. 通常処理：「条件10を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.4.1. 条件成立時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

5.5. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

5-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

5-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

5-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

結果・例外時の扱い

到達条件	結果
code が等しい 0	戻り先画面を指定せず処理を終了する
NOT (code が等しい 0)・条件11を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (code が等しい 0)・条件12を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す

exists

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「exists」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理4：詳細仕様参照）。
2. 関連情報を取得・設定する（処理5：詳細仕様参照）。
3. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

3.1. 通常処理：「 WHERE ふりがな = ? AND 電話番号 = ?」を設定する。

3.2. 通常処理：「対象項目」を設定する。

3.3. 通常処理：「対象項目」を設定する。

3.4. 通常処理：関連情報を取得・設定する（処理6：詳細仕様参照）。

3.5. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

3-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

3-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

3-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-19BE5BBBA554

情報	内容
操作ID	SQL-19BE5BBBA554
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:628 / exists@623

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

findByKey

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「findByKey」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理4：詳細仕様参照）。
2. 関連情報を取得・設定する（処理5：詳細仕様参照）。
3. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

3.1. 通常処理：「 WHERE R.ふりがな = ? AND R.電話番号 = ?」を設定する。

3.2. 通常処理：「対象項目」を設定する。

3.3. 通常処理：「対象項目」を設定する。

3.4. 通常処理：関連情報を取得・設定する（処理6：詳細仕様参照）。

3.5. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

3-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

3-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

3-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

insert

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「insert」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理4：詳細仕様参照）。
2. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理5：詳細仕様参照）。

2.2. 通常処理：関連情報を取得・設定する（処理6：詳細仕様参照）。

2.3. 通常処理：「statement.executeUpdate() が異なる 1」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.3.1. 条件成立時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

2-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

2-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ＝?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-1A463E2BC06C

情報	内容
操作ID	SQL-1A463E2BC06C
対象テーブル	年金1_LK
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:661 / insert@657

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, Integer.valueOf(0))

結果・例外時の扱い

到達条件	結果
statement.executeUpdate() が異なる 1	処理の異常を通知し、この経路を終了する

update

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「update」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理5：詳細仕様参照）。
2. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理6：詳細仕様参照）。

2.2. 通常処理：関連情報を取得・設定する（処理7：詳細仕様参照）。

2.3. 通常処理：「対象項目」を設定する。

2.4. 通常処理：「対象項目」を設定する。

2.5. 通常処理：関連情報を取得・設定する（処理8：詳細仕様参照）。

2.6. 通常処理：「changed が異なる 1」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.6.1. 条件成立時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

2-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

2-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-CD4068C5AE3B

情報	内容
操作ID	SQL-CD4068C5AE3B
対象テーブル	年金1_LK
操作	更新
処理説明	対象テーブルの条件に一致する行を更新する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:676 / update@672

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, Integer.valueOf(1))

結果・例外時の扱い

到達条件	結果
changed が異なる 1	処理の異常を通知し、この経路を終了する

insertDeleted

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「insertDeleted」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理4：詳細仕様参照）。
2. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理5：詳細仕様参照）。

2.2. 通常処理：関連情報を取得・設定する（処理6：詳細仕様参照）。

2.3. 通常処理：「statement.executeUpdate() が異なる 1」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.3.1. 条件成立時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

2-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

2-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
statement.executeUpdate() が異なる 1	処理の異常を通知し、この経路を終了する

bindRecord

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「bindRecord」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目」を設定する。
2. 「対象項目」を設定する。
3. 「対象項目」を設定する。
4. 「対象項目」を設定する。
5. 「対象項目」を設定する。
6. 「対象項目」を設定する。
7. 「対象項目」を設定する。
8. 「対象項目」を設定する。
9. 「対象項目」を設定する。
10. 「対象項目」を設定する。
11. 「対象項目」を設定する。
12. 「対象項目」を設定する。
13. 「対象項目」を設定する。
14. 「対象項目」を設定する。
15. 「対象項目」を設定する。
16. 「対象項目」を設定する。
17. 「対象項目」を設定する。

18. 「対象項目」を設定する。
19. 「対象項目」を設定する。
20. 「対象項目」を設定する。
21. 「対象項目」を設定する。
22. 「対象項目」を設定する。
23. 「対象項目」を設定する。
24. 「対象項目」を設定する。
25. 「対象項目」を設定する。
26. 「対象項目」を設定する。
27. 「対象項目」を設定する。
28. 「対象項目」を設定する。
29. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。

情報	内容
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

readReservation

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「readReservation」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 「対象項目」を設定する。
3. 「店舗名」を設定する。
4. 「対象項目」を設定する。
5. 「対象項目」を設定する。

6. 「対象項目」を設定する。
7. 「対象項目」を設定する。
8. 「対象項目」を設定する。
9. 「対象項目」を設定する。
10. 「対象項目」を設定する。
11. 「対象項目」を設定する。
12. 「対象項目」を設定する。
13. 「対象項目」を設定する。
14. 「対象項目」を設定する。
15. 「対象項目」を設定する。
16. 「対象項目」を設定する。
17. 「対象項目」を設定する。
18. 「対象項目」を設定する。
19. 「対象項目」を設定する。
20. 「対象項目」を設定する。
21. 「年金種別」を設定する。
22. 「対象項目」を設定する。
23. 「対象項目」を設定する。
24. 「対象項目」を設定する。
25. 「対象項目」を設定する。
26. 「対象項目」を設定する。
27. 「対象項目」を設定する。
28. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

fromClause

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「fromClause」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. エラーまたは処理結果「 FROM " + table + " R" + " LEFT JOIN " + BRANCH_TABLE + " B ON R.店番 = B.店舗ｺｰﾄﾞ" + " LEFT JOIN " + GENDER_TABLE + " G ON R.性別 = G.性別ｺｰﾄﾞ" + " LEFT JOIN " + CLAIM_TABLE + " C ON R.裁定請求 = C.裁請ｺｰﾄﾞ" + " LEFT JOIN " + BENEFIT_TABLE + " H ON R.受給有無 = H.受給ｺｰﾄﾞ" + " LEFT JOIN " + PENSION_TYPE_TABLE + " P ON R.年金種類 = P.種類ｺｰﾄﾞ」を返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	エラーまたは処理結果「 FROM " + table + " R" + " LEFT JOIN " + BRANCH_TABLE + " B ON R.店番 = B.店舗ｺｰﾄﾞ" + " LEFT JOIN " + GENDER_TABLE + " G ON R.性別 = G.性別ｺｰﾄﾞ" + " LEFT JOIN " + CLAIM_TABLE + " C ON R.裁定請求 = C.裁請ｺｰﾄﾞ" + " LEFT JOIN " + BENEFIT_TABLE + " H ON R.受給有無 = H.受給ｺｰﾄﾞ" + " LEFT JOIN " + PENSION_TYPE_TABLE + " P ON R.年金種類 = P.種類ｺｰﾄﾞ」を返す

nullableInteger

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「nullableInteger」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

integerOrNull

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「integerOrNull」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 「text.length() が等しい 0」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：戻り先画面を指定せず処理を終了する。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

3.1. 通常処理：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

3-例外1. 例外条件「NumberFormatException e」に該当した場合に次の手順を行う。型名は原本の表記。

3-例外1.1. 例外発生時：戻り先画面を指定せず処理を終了する。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
text.length() が等しい 0	戻り先画面を指定せず処理を終了する
NOT (text.length() が等しい 0)	処理結果を呼出元へ返す
NOT (text.length() が等しい 0)・条件3を満たす（原本条件は補足説明を参照）	戻り先画面を指定せず処理を終了する

setInteger

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「setInteger」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1. 条件成立時：「対象項目」を設定する。

1-別.1. 条件不成立時：「対象項目」を設定する。

setPhone

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「setPhone」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 「対象項目がない」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2.1. 条件成立時：「対象項目」を設定する。

2-別.1. 条件不成立時：「対象項目」を設定する。

setDate

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「setDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 「date.length() が等しい 0」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2.1. 条件成立時：「対象項目」を設定する。

2-別.1. 条件不成立時：「対象項目」を設定する。

setTime

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「setTime」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理3：詳細仕様参照）。
2. 「条件4を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2.1. 条件成立時：「対象項目」を設定する。

2-別.1. 条件不成立時：「対象項目」を設定する。

normalize

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「normalize」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

digits

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「digits」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理9：詳細仕様参照）。
2. 関連情報を取得・設定する（処理10：詳細仕様参照）。
3. 繰返し条件「条件11を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

3.1. 繰返しの各回：関連情報を取得・設定する（処理12：詳細仕様参照）。

3.2. 繰返しの各回：「条件13を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.2.1. 条件成立時：関連情報を取得・設定する（処理14：詳細仕様参照）。

3.2.2. 条件成立時：「条件15を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.2.2.1. 条件成立時：関連情報を取得・設定する（処理16：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

phoneValue

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「phoneValue」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

canonicalPhone

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「canonicalPhone」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

optionalDate

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「optionalDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

text

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「text」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

join

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「join」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 繰返し条件「条件7を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：「条件8を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1.1. 条件成立時：関連情報を取得・設定する（処理9：詳細仕様参照）。

2.2. 繰返しの各回：関連情報を取得・設定する（処理10：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

placeholders

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「placeholders」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 繰返し条件「条件7を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：「条件8を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1.1. 条件成立時：関連情報を取得・設定する（処理9：詳細仕様参照）。

2.2. 繰返しの各回：関連情報を取得・設定する（処理10：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

assignments

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「assignments」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 繰返し条件「条件7を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：「条件8を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1.1. 条件成立時：関連情報を取得・設定する（処理9：詳細仕様参照）。

2.2. 繰返しの各回：関連情報を取得・設定する（処理10：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

containsPattern

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「containsPattern」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

failure

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「failure」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

rollback

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「rollback」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：このトランザクションの変更を取り消す（ロールバック）。

2-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

結果・例外時の扱い

到達条件	結果
対象項目がない	処理結果を呼出元へ返す

close

処理概要

Access と同じ年金業務表を N2 JNDI 経由で操作する Repositoryで「close」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：使用した接続・処理資源を解放する。

2-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

結果・例外時の扱い

到達条件	結果
対象項目がない	処理結果を呼出元へ返す

DB関連処理

補足説明

適用条件

各仮定のQA番号を確認する。静的に確認できた操作・条件を適用する。

処理・条件の原本対応

文書内の条件番号・処理番号は次の原本表記に対応する。完全な業務用語への置換を行っていない箇所はQA・reportで確認する。

イベント	参照番号	原本条件・処理
findByCustomer	処理1	businessConnection = NenkinDataSource.openConnection()
findByCustomer	処理2	Connection businessConnection = null
findByCustomer	処理3	genderCode = text(results.getString("GENDER_CODE")).trim()
findByCustomer	処理4	value = new PensionReservation()
findByCustomer	条件5	results.next()

イベント	参照番号	原本条件・処理
findByCustomer	処理6	results = statement.executeQuery()
findByCustomer	処理7	statement = customerConnection.prepareStatement(sql)
findByCustomer	処理8	customerConnection = NenkinDataSource.openCustomerConnection()
findByCustomer	処理9	String genderCode = ""
findByCustomer	処理10	PensionReservation value = null
findByCustomer	処理11	ResultSet results = null
findByCustomer	処理12	PreparedStatement statement = null
findByCustomer	処理13	Connection customerConnection = null
findByCustomer	処理14	Connection customerConnection = null
findByCustomer	処理15	PreparedStatement statement = null
findByCustomer	処理16	ResultSet results = null
findByCustomer	処理17	PensionReservation value = null
findByCustomer	処理18	String genderCode = ""
findByCustomer	処理19	customerConnection = NenkinDataSource.openCustomerConnection()
findByCustomer	処理20	statement = customerConnection.prepareStatement(sql)
findByCustomer	処理21	results = statement.executeQuery()
findByCustomer	条件22	results.next()
findByCustomer	処理23	value = new PensionReservation()

イベント	参照番号	原本条件・処理
findByCustomer	処理24	genderCode = text(results.getString("GENDER_CODE")).trim()
findByCustomer	処理25	Connection businessConnection = null
findByCustomer	処理26	businessConnection = NenkinDataSource.openConnection()
findByCustomer	条件27	SQLException e
findByCustomer	条件28	SQLException e
findByKanaAndPhone	処理1	results = statement.executeQuery()
findByKanaAndPhone	処理2	statement = connection.prepareStatement(sql)
findByKanaAndPhone	処理3	connection = NenkinDataSource.openConnection()
findByKanaAndPhone	処理4	ResultSet results = null
findByKanaAndPhone	処理5	PreparedStatement statement = null
findByKanaAndPhone	処理6	Connection connection = null
findByKanaAndPhone	処理7	BigDecimal phone = phoneValue(phoneNumber)
findByKanaAndPhone	処理8	String kana = normalize(kanaName)
findByKanaAndPhone	処理9	String kana = normalize(kanaName)
findByKanaAndPhone	処理10	BigDecimal phone = phoneValue(phoneNumber)
findByKanaAndPhone	処理11	Connection connection = null
findByKanaAndPhone	処理12	PreparedStatement statement = null
findByKanaAndPhone	処理13	ResultSet results = null
findByKanaAndPhone	処理14	connection = NenkinDataSource.openConnection()

イベント	参照番号	原本条件・処理
findByKanaAndPhone	処理15	statement = connection.prepareStatement(sql)
findByKanaAndPhone	処理16	results = statement.executeQuery()
findByKanaAndPhone	条件17	SQLException e
searchCandidates	条件1	results.next()
searchCandidates	処理2	results = statement.executeQuery()
searchCandidates	条件3	phone.length() > 0
searchCandidates	条件4	kana.length() > 0
searchCandidates	処理5	int index = 1
searchCandidates	処理6	statement = connection.prepareStatement(sql.toString())
searchCandidates	処理7	connection = NenkinDataSource.openConnection()
searchCandidates	処理8	ResultSet results = null
searchCandidates	処理9	PreparedStatement statement = null
searchCandidates	処理10	Connection connection = null
searchCandidates	処理11	List<PensionCandidate> rows = new ArrayList<PensionCandidate>()
searchCandidates	条件12	phone.length() > 0
searchCandidates	条件13	kana.length() > 0
searchCandidates	処理14	String phone = canonicalPhone(phoneNumber)
searchCandidates	処理15	String kana = normalize(kanaName)
searchCandidates	処理16	String kana = normalize(kanaName)

イベント	参照番号	原本条件・処理
searchCandidates	処理17	String phone = canonicalPhone(phoneNumber)
searchCandidates	条件18	kana.length() > 0
searchCandidates	条件19	phone.length() > 0
searchCandidates	処理20	List<PensionCandidate> rows = new ArrayList<PensionCandidate>()
searchCandidates	処理21	Connection connection = null
searchCandidates	処理22	PreparedStatement statement = null
searchCandidates	処理23	ResultSet results = null
searchCandidates	処理24	connection = NenkinDataSource.openConnection()
searchCandidates	処理25	statement = connection.prepareStatement(sql.toString())
searchCandidates	処理26	int index = 1
searchCandidates	条件27	kana.length() > 0
searchCandidates	条件28	phone.length() > 0
searchCandidates	処理29	results = statement.executeQuery()
searchCandidates	条件30	results.next()
searchCandidates	条件31	SQLException e
listByBranch	条件1	results.next()
listByBranch	処理2	rows.add(readReservation(results))
listByBranch	処理3	results = statement.executeQuery()
listByBranch	処理4	statement = connection.prepareStatement(sql)

イベント	参照番号	原本条件・処理
listByBranch	処理5	connection = NenkinDataSource.openConnection()
listByBranch	処理6	ResultSet results = null
listByBranch	処理7	PreparedStatement statement = null
listByBranch	処理8	Connection connection = null
listByBranch	処理9	List<PensionReservation> rows = new ArrayList<PensionReservation>()
listByBranch	処理10	List<PensionReservation> rows = new ArrayList<PensionReservation>()
listByBranch	処理11	Connection connection = null
listByBranch	処理12	PreparedStatement statement = null
listByBranch	処理13	ResultSet results = null
listByBranch	処理14	connection = NenkinDataSource.openConnection()
listByBranch	処理15	statement = connection.prepareStatement(sql)
listByBranch	処理16	results = statement.executeQuery()
listByBranch	条件17	results.next()
listByBranch	処理18	rows.add(readReservation(results))
listByBranch	条件19	SQLException e
search	例外	sortType
search	処理2	StringBuilder sql = new StringBuilder("SELECT ").append(SELECT_COLUMNS) .append(fromClause(TABLE)).append(" WHERE 1 = 1")

イベント	参照番号	原本条件・処理
search	処理3	StringBuilder sql = new StringBuilder("SELECT ").append(SELECT_COLUMNS) .append(fromClause(TABLE)).append(" WHERE 1 = 1")
save	処理1	Integer updatedBy = reservation == null ? null : reservation.getStaffCode()
save	処理2	Integer updatedBy = reservation == null ? null : reservation.getStaffCode()
save	条件1	e instanceof NenkinDataAccessException
save	条件2	exists
save	処理3	String updateTime = DateUtil.currentTime()
save	処理4	AuditType type = exists ? AuditType.CHANGED : AuditType.NEW
save	処理5	boolean exists = exists(connection, keyKana, keyPhone)
save	処理6	connection = NenkinDataSource.openConnection()
save	処理7	Connection connection = null
save	処理8	BigDecimal keyPhone = phoneValue(value.getPhoneNumber())
save	処理9	String keyKana = value.getKanaName()
save	処理10	PensionReservation value = prepareForSave(reservation)
save	処理11	PensionReservation value = prepareForSave(reservation)
save	処理12	String keyKana = value.getKanaName()
save	処理13	BigDecimal keyPhone = phoneValue(value.getPhoneNumber())
save	処理14	Connection connection = null

イベント	参照番号	原本条件・処理
save	処理15	connection = NenkinDataSource.openConnection()
save	処理16	boolean exists = exists(connection, keyKana, keyPhone)
save	処理17	AuditType type = exists ? AuditType.CHANGED : AuditType.NEW
save	処理18	String updateTime = DateUtil.currentTime()
save	条件19	exists
save	条件20	e instanceof NenkinDataAccessException
save	条件21	Exception e
save	条件22	e instanceof NenkinDataAccessException
save	条件23	Exception e
save	条件24	NOT (e instanceof NenkinDataAccessException)
delete	処理1	int deleted = delete.executeUpdate()
delete	処理2	insertDeleted(connection, existing, updatedBy, DateUtil.currentTime())
delete	処理3	connection = NenkinDataSource.openConnection()
delete	処理4	PreparedStatement delete = null
delete	処理5	Connection connection = null
delete	処理6	BigDecimal phone = phoneValue(phoneNumber)
delete	処理7	String kana = normalize(kanaName)
delete	処理8	String kana = normalize(kanaName)
delete	処理9	BigDecimal phone = phoneValue(phoneNumber)

イベント	参照番号	原本条件・処理
delete	処理10	Connection connection = null
delete	処理11	PreparedStatement delete = null
delete	処理12	connection = NenkinDataSource.openConnection()
delete	処理13	insertDeleted(connection, existing, updatedBy, DateUtil.currentTime())
delete	処理14	int deleted = delete.executeUpdate()
delete	条件15	Exception e
findAuditRecords	条件1	results.next()
findAuditRecords	処理2	rows.add(new PensionAuditRecord(rowId++, type, results.getString("AUDIT_DATE"), results.getString("AUDIT_TIME"), nullableInteger(results, "UPDATED_BY"), readReservation(results)))
findAuditRecords	処理3	long rowId = 1L
findAuditRecords	処理4	results = statement.executeQuery()
findAuditRecords	条件5	to.length() > 0
findAuditRecords	条件6	from.length() > 0
findAuditRecords	処理7	int index = 1
findAuditRecords	処理8	statement = connection.prepareStatement(sql.toString())
findAuditRecords	処理9	connection = NenkinDataSource.openConnection()
findAuditRecords	処理10	ResultSet results = null
findAuditRecords	処理11	PreparedStatement statement = null

イベント	参照番号	原本条件・処理
findAuditRecords	処理12	Connection connection = null
findAuditRecords	処理13	List<PensionAuditRecord> rows = new ArrayList<PensionAuditRecord>()
findAuditRecords	条件14	to.length() > 0
findAuditRecords	条件15	from.length() > 0
findAuditRecords	処理16	String sourceTable = type == AuditType.DELETED ? DELETE_TABLE : TABLE
findAuditRecords	処理17	String to = optionalDate(dateTo)
findAuditRecords	処理18	String from = optionalDate(dateFrom)
findAuditRecords	処理19	String from = optionalDate(dateFrom)
findAuditRecords	処理20	String to = optionalDate(dateTo)
findAuditRecords	処理21	String sourceTable = type == AuditType.DELETED ? DELETE_TABLE : TABLE
findAuditRecords	条件22	from.length() > 0
findAuditRecords	条件23	to.length() > 0
findAuditRecords	処理24	List<PensionAuditRecord> rows = new ArrayList<PensionAuditRecord>()
findAuditRecords	処理25	Connection connection = null
findAuditRecords	処理26	PreparedStatement statement = null
findAuditRecords	処理27	ResultSet results = null
findAuditRecords	処理28	connection = NenkinDataSource.openConnection()

イベント	参照番号	原本条件・処理
findAuditRecords	処理29	statement = connection.prepareStatement(sql.toString())
findAuditRecords	処理30	int index = 1
findAuditRecords	条件31	from.length() > 0
findAuditRecords	条件32	to.length() > 0
findAuditRecords	処理33	results = statement.executeQuery()
findAuditRecords	処理34	long rowId = 1L
findAuditRecords	条件35	results.next()
findAuditRecords	処理36	rows.add(new PensionAuditRecord(rowId++, type, results.getString("AUDIT_DATE"), results.getString("AUDIT_TIME"), nullableInteger(results, "UPDATED_BY"), readReservation(results)))
findAuditRecords	条件37	SQLException e
findLatestAuditType	処理1	results = statement.executeQuery()
findLatestAuditType	処理2	results = null
findLatestAuditType	処理3	int flag = results.getInt(1)
findLatestAuditType	条件4	results.next()
findLatestAuditType	処理5	results = statement.executeQuery()
findLatestAuditType	処理6	connection = NenkinDataSource.openConnection()
findLatestAuditType	処理7	ResultSet results = null
findLatestAuditType	処理8	PreparedStatement statement = null
findLatestAuditType	処理9	Connection connection = null

イベント	参照番号	原本条件・処理
findLatestAuditType	処理10	BigDecimal phone = phoneValue(phoneNumber)
findLatestAuditType	処理11	String kana = normalize(kanaName)
findLatestAuditType	処理12	String kana = normalize(kanaName)
findLatestAuditType	処理13	BigDecimal phone = phoneValue(phoneNumber)
findLatestAuditType	処理14	Connection connection = null
findLatestAuditType	処理15	PreparedStatement statement = null
findLatestAuditType	処理16	ResultSet results = null
findLatestAuditType	処理17	connection = NenkinDataSource.openConnection()
findLatestAuditType	処理18	results = statement.executeQuery()
findLatestAuditType	条件19	results.next()
findLatestAuditType	処理20	int flag = results.getInt(1)
findLatestAuditType	処理21	results = null
findLatestAuditType	処理22	results = statement.executeQuery()
findLatestAuditType	条件23	results.next()
findLatestAuditType	条件24	NOT (results.next())
findLatestAuditType	条件25	SQLException e
purgeDeletedAuditsThrough	処理1	int count = statement.executeUpdate()
purgeDeletedAuditsThrough	処理2	connection = NenkinDataSource.openConnection()
purgeDeletedAuditsThrough	処理3	PreparedStatement statement = null

イベント	参照番号	原本条件・処理
purgeDeletedAuditsThrough	処理4	Connection connection = null
purgeDeletedAuditsThrough	処理5	String cutoff = optionalDate(cutoffDate)
purgeDeletedAuditsThrough	処理6	String cutoff = optionalDate(cutoffDate)
purgeDeletedAuditsThrough	処理7	Connection connection = null
purgeDeletedAuditsThrough	処理8	PreparedStatement statement = null
purgeDeletedAuditsThrough	処理9	connection = NenkinDataSource.openConnection()
purgeDeletedAuditsThrough	処理10	int count = statement.executeUpdate()
purgeDeletedAuditsThrough	条件11	SQLException e
queryReservations	条件1	results.next()
queryReservations	処理2	rows.add(readReservation(results))
queryReservations	処理3	results = statement.executeQuery()
queryReservations	処理4	int index = 1
queryReservations	処理5	statement = connection.prepareStatement(sql)
queryReservations	処理6	connection = NenkinDataSource.openConnection()
queryReservations	処理7	ResultSet results = null
queryReservations	処理8	PreparedStatement statement = null
queryReservations	処理9	Connection connection = null
queryReservations	処理10	List<PensionReservation> rows = new ArrayList<PensionReservation>()

イベント	参照番号	原本条件・処理
queryReservations	処理11	List<PensionReservation> rows = new ArrayList<PensionReservation>()
queryReservations	処理12	Connection connection = null
queryReservations	処理13	PreparedStatement statement = null
queryReservations	処理14	ResultSet results = null
queryReservations	処理15	connection = NenkinDataSource.openConnection()
queryReservations	処理16	statement = connection.prepareStatement(sql)
queryReservations	処理17	int index = 1
queryReservations	処理18	results = statement.executeQuery()
queryReservations	条件19	results.next()
queryReservations	処理20	rows.add(readReservation(results))
queryReservations	条件21	SQLException e
prepareForSave	処理1	PensionReservation value = source.copy()
prepareForSave	処理2	PensionReservation value = source.copy()
resolveBranchName	処理1	results = statement.executeQuery()
resolveBranchName	処理2	ResultSet results = null
resolveBranchName	処理3	PreparedStatement statement = null
resolveBranchName	処理4	PreparedStatement statement = null
resolveBranchName	処理5	ResultSet results = null
resolveBranchName	処理6	results = statement.executeQuery()

イベント	参照番号	原本条件・処理
resolveCodeLabel	処理1	results = statement.executeQuery()
resolveCodeLabel	処理2	statement = connection.prepareStatement("SELECT " + labelColumn + " FROM " + table + " WHERE " + codeColumn + " = ?")
resolveCodeLabel	処理3	ResultSet results = null
resolveCodeLabel	処理4	PreparedStatement statement = null
resolveCodeLabel	処理5	PreparedStatement statement = null
resolveCodeLabel	処理6	ResultSet results = null
resolveCodeLabel	処理7	statement = connection.prepareStatement("SELECT " + labelColumn + " FROM " + table + " WHERE " + codeColumn + " = ?")
resolveCodeLabel	処理8	results = statement.executeQuery()
resolveCode	条件1	!results.next()
resolveCode	処理2	results = statement.executeQuery()
resolveCode	処理3	statement = connection.prepareStatement("SELECT " + codeColumn + " FROM " + table + " WHERE " + labelColumn + " = ?")
resolveCode	処理4	ResultSet results = null
resolveCode	処理5	PreparedStatement statement = null
resolveCode	処理6	String value = text(label).trim()
resolveCode	処理7	String value = text(label).trim()
resolveCode	処理8	PreparedStatement statement = null

イベント	参照番号	原本条件・処理
resolveCode	処理9	ResultSet results = null
resolveCode	処理10	statement = connection.prepareStatement("SELECT " + codeColumn + " FROM " + table + " WHERE " + labelColumn + " = ?")
resolveCode	処理11	results = statement.executeQuery()
resolveCode	条件12	!results.next()
resolveCode	条件13	!results.next()
resolveCode	条件14	NOT (!results.next())
resolvePensionTypeCode	条件1	!results.next()
resolvePensionTypeCode	処理2	results = statement.executeQuery()
resolvePensionTypeCode	処理3	ResultSet results = null
resolvePensionTypeCode	処理4	PreparedStatement statement = null
resolvePensionTypeCode	処理5	int code = (value.isNationalPension() ? 1 : 0) + (value.isEmployeePension() ? 2 : 0) + (value.isMutualAidPension() ? 4 : 0) + (value.isEmployeePensionFund() ? 8 : 0) + (value.isNationalPensionFund() ? 16 : 0)
resolvePensionTypeCode	処理6	int code = (value.isNationalPension() ? 1 : 0) + (value.isEmployeePension() ? 2 : 0) + (value.isMutualAidPension() ? 4 : 0) + (value.isEmployeePensionFund() ? 8 : 0) + (value.isNationalPensionFund() ? 16 : 0)
resolvePensionTypeCode	処理7	PreparedStatement statement = null
resolvePensionTypeCode	処理8	ResultSet results = null
resolvePensionTypeCode	処理9	results = statement.executeQuery()

イベント	参照番号	原本条件・処理
resolvePensionTypeCode	条件10	!results.next()
resolvePensionTypeCode	条件11	!results.next()
resolvePensionTypeCode	条件12	NOT (!results.next())
exists	処理1	results = statement.executeQuery()
exists	処理2	ResultSet results = null
exists	処理3	PreparedStatement statement = null
exists	処理4	PreparedStatement statement = null
exists	処理5	ResultSet results = null
exists	処理6	results = statement.executeQuery()
findByKey	処理1	results = statement.executeQuery()
findByKey	処理2	ResultSet results = null
findByKey	処理3	PreparedStatement statement = null
findByKey	処理4	PreparedStatement statement = null
findByKey	処理5	ResultSet results = null
findByKey	処理6	results = statement.executeQuery()
insert	処理1	bindRecord(connection, statement, 1, value, updatedBy, updateTime, Integer.valueOf(0))
insert	処理2	statement = connection.prepareStatement(INSERT_SQL)
insert	処理3	PreparedStatement statement = null
insert	処理4	PreparedStatement statement = null

イベント	参照番号	原本条件・処理
insert	処理5	statement = connection.prepareStatement(INSERT_SQL)
insert	処理6	bindRecord(connection, statement, 1, value, updatedBy, updateTime, Integer.valueOf(0))
update	処理1	int changed = statement.executeUpdate()
update	処理2	int index = bindRecord(connection, statement, 1, value, updatedBy, updateTime, Integer.valueOf(1))
update	処理3	statement = connection.prepareStatement(UPDATE_SQL)
update	処理4	PreparedStatement statement = null
update	処理5	PreparedStatement statement = null
update	処理6	statement = connection.prepareStatement(UPDATE_SQL)
update	処理7	int index = bindRecord(connection, statement, 1, value, updatedBy, updateTime, Integer.valueOf(1))
update	処理8	int changed = statement.executeUpdate()
insertDeleted	処理1	bindRecord(connection, statement, 1, value, updatedBy, updateTime, null)
insertDeleted	処理2	statement = connection.prepareStatement(INSERT_DELETED_SQL)
insertDeleted	処理3	PreparedStatement statement = null
insertDeleted	処理4	PreparedStatement statement = null
insertDeleted	処理5	statement = connection.prepareStatement(INSERT_DELETED_SQL)

イベント	参照番号	原本条件・処理
insertDeleted	処理6	bindRecord(connection, statement, 1, value, updatedBy, updateTime, null)
readReservation	処理1	PensionReservation value = new PensionReservation()
readReservation	処理2	PensionReservation value = new PensionReservation()
nullableInteger	処理1	int value = results.getInt(column)
nullableInteger	処理2	int value = results.getInt(column)
integerOrNull	処理1	String text = value == null ? "" : value.trim()
integerOrNull	処理2	String text = value == null ? "" : value.trim()
integerOrNull	条件3	NumberFormatException e
setPhone	処理1	BigDecimal phone = phoneValue(value)
setPhone	処理2	BigDecimal phone = phoneValue(value)
setDate	処理1	String date = text(value).trim()
setDate	処理2	String date = text(value).trim()
setTime	条件1	time.length() < 8
setTime	処理2	String time = text(value)
setTime	処理3	String time = text(value)
setTime	条件4	time.length() < 8
digits	条件1	int i = 0; i < source.length(); i++
digits	処理2	result.append((char) ('0' + digit))
digits	条件3	digit >= 0

イベント	参照番号	原本条件・処理
digits	処理4	int digit = Character.digit(character, 10)
digits	条件5	Character.isDigit(character)
digits	処理6	char character = source.charAt(i)
digits	処理7	StringBuilder result = new StringBuilder(source.length())
digits	処理8	String source = text(value)
digits	処理9	String source = text(value)
digits	処理10	StringBuilder result = new StringBuilder(source.length())
digits	条件11	int i = 0; i < source.length(); i++
digits	処理12	char character = source.charAt(i)
digits	条件13	Character.isDigit(character)
digits	処理14	int digit = Character.digit(character, 10)
digits	条件15	digit >= 0
digits	処理16	result.append((char) ('0' + digit))
phoneValue	処理1	String phone = digits(value)
phoneValue	処理2	String phone = digits(value)
canonicalPhone	処理1	BigDecimal phone = phoneValue(value)
canonicalPhone	処理2	BigDecimal phone = phoneValue(value)
join	条件1	int i = 0; i < values.length; i++
join	処理2	result.append(values[i])

イベント	参照番号	原本条件・処理
join	処理3	result.append(separator)
join	条件4	i > 0
join	処理5	StringBuilder result = new StringBuilder()
join	処理6	StringBuilder result = new StringBuilder()
join	条件7	int i = 0; i < values.length; i++
join	条件8	i > 0
join	処理9	result.append(separator)
join	処理10	result.append(values[i])
placeholders	条件1	int i = 0; i < count; i++
placeholders	処理2	result.append('?')
placeholders	処理3	result.append(", ")
placeholders	条件4	i > 0
placeholders	処理5	StringBuilder result = new StringBuilder()
placeholders	処理6	StringBuilder result = new StringBuilder()
placeholders	条件7	int i = 0; i < count; i++
placeholders	条件8	i > 0
placeholders	処理9	result.append(", ")
placeholders	処理10	result.append('?')
assignments	条件1	int i = 0; i < values.length; i++

イベント	参照番号	原本条件・処理
assignments	処理2	result.append(values[i]).append(" = ?")
assignments	処理3	result.append(", ")
assignments	条件4	i > 0
assignments	処理5	StringBuilder result = new StringBuilder()
assignments	処理6	StringBuilder result = new StringBuilder()
assignments	条件7	int i = 0; i < values.length; i++
assignments	条件8	i > 0
assignments	処理9	result.append(", ")
assignments	処理10	result.append(values[i]).append(" = ?")
containsPattern	処理1	String escaped = text(value).replace("\", "\\") .replace("%", "\%").replace("", "\") .replace("[", "\[").replace("]", "\]")
containsPattern	処理2	String escaped = text(value).replace("\", "\\") .replace("%", "\%").replace("", "\") .replace("[", "\[").replace("]", "\]")